Latest updates: https://dl.acm.org/doi/10.1145/3084100.3084107

RESEARCH-ARTICLE

Rapid, evolutionary, reliable, scalable system and software development: the resilient agile process

DOUG ROSENBERG, University of Southern California, Los Angeles, CA, United States

BARRY WILLIAM BOEHM, University of Southern California, Los Angeles, CA, United States

BO WANG, University of Southern California, Los Angeles, CA, United States

KAN QI, University of Southern California, Los Angeles, CA, United States

Open Access Support provided by:

University of Southern California

PDF Download
3084100.3084107.pdf
23 January 2026
Total Citations: 11
Total Downloads: 585

Published: 05 July 2017

Citation in BibTeX format

ICSSP 2017: International Conference on the Software and Systems Process 2017
July 5 - 7, 2017
Paris, France

ICSSP '17: Proceedings of the 2017 International Conference on Software and System Process (July 2017)
https://doi.org/10.1145/3084100.3084107
ISBN: 9781450352703

Rapid, Evolutionary, Reliable, Scalable System and Software Development: The Resilient Agile Process

Doug Rosenberg
ICONIX Software Engineering and USC
United States
resilientagile@iconixsw.com

Barry W. Boehm
University of Southern California
United States
boehm@usc.edu

Bo Wang
University of Southern California
United States
wang736@usc.edu

Kan Qi
University of Southern California
United States
kqi@usc.edu

ABSTRACT

The increasing pace of change in competition, technology, and complexity of software-intensive systems has increased the demand for rapid, reliable, scalable, and evolvable processes. Agile methods have made significant contributions to speeding up software development, but often encounter problems with reliability, scalability, and evolvability. Over the past 3 years, we have been experimenting with an approach called Resilient Agile (RA), which addresses these problems while also speeding up development by finding enablers for parallel systems engineering, development, and test. This paper summarizes our experience in defining and evolving RA by applying it to three representative emergent-technology applications: Location-Based Advertising, Picture Sharing, and Bad Driver Reporting. In comparison with the mainstream Architected Agile process that we had been using on similar systems, the RA process achieved fewer defects and significant speedups in system development and evolution.

The paper summarizes the overall challenge of software schedule compression; identifies managed parallel development as generally the most powerful but least-practiced strategy for schedule compression; summarizes the key elements required to support parallelism, including specific model-driven system development techniques, automatic generation of key elements and realistic schedule and effort estimation. It then summarizes the three successful Resilient Agile projects to date, provides criteria for selecting a Resilient Agile process, and summarizes the key techniques for scaling up Resilient Agile, using a previous million-line command and control project as an example.

CCS CONCEPTS

• **Software and its engineering → Agile software development**; **Rapid application development**; *Software architectures*; *Model-driven software engineering*; *Spiral model*; *Software prototyping*;

KEYWORDS

agile development, schedule compression, rapid delivery, NoSQL, REST, code generation, MVC, parallel development, scalable software development, ICSM, microservice architecture, resilient software, UML modeling, use case driven development

ACM Reference format:
Doug Rosenberg, Barry W. Boehm, Bo Wang, and Kan Qi. 2017. Rapid, Evolutionary, Reliable, Scalable System and Software Development: The Resilient Agile Process. In *Proceedings of 2017 International Conference on Software and Systems Process , Paris, France, July 2017 (ICSSP'17),* 10 pages.
DOI: 10.1145/3084100.3084107

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICSSP'17, Paris, France
© 2017 ACM. 978-1-4503-5270-3/17/07…$15.00
DOI: 10.1145/3084100.3084107

1 INTRODUCTION

Our continuing interactions with our government and industry affiliates have indicated that earlier forms of agile methods can be rapid, but often achieve speed by addressing only the sunny-day use cases, causing serious reliability problems. Some progress in improving agile development reliability has resulted from the Architected Agile[1] process, but often the build-to architecture remains fixed and non-responsive as the marketplace and technology evolve. Approaches such as Scrum and Kanban address such problems, but tend to focus on project management and less on design for parallel development.

For the last 3 years, we have been experimenting with the Resilient Agile approach. It involves a highly skilled Systems Engineering team exploring options with clients via storyboards, workflows, and prototypes to define both sunny-day and rainy-day use cases; defining test requirements and plans for each usage scenario; decomposing each use case into a conceptual Model-View-Controller (MVC) pattern, and continuing to evolve the architecture as client needs and opportunities evolve.

Resilient Agile achieves significant schedule compression by leveraging parallelism; as many developers as necessary can independently and concurrently develop the scenarios from initial concept through code. Further, a system test team can use the MVC decompositions to quickly generate a test suite to support continuous testing of the evolving system. We have used this approach to develop a working Location-Based Advertising system (LBA)

involving more than 70 developers overall and 29 parallel scenario developers, a working Picture Sharing system (PicShare) involving 10 scenario developers, and a proof-of-concept Bad Driver Reporting system (BDR) involving 15 scenario developers to-date (ongoing project). In each case, the scenario developer effort has been roughly 80 hours, or a 2-week time commitment.

Resilient Agile is fully compatible with the Incremental Commitment Spiral Model (ICSM)[2], which involves a 3 team concurrent effort of a Systems Engineering team, a Development team, and a dedicated System Test team. The stabilization afforded by Resilient Agile's "conceptual MVC" scenario decomposition enables the Development team to operate in a highly parallel mode, enabling more rapid development and evolution while benefiting from the ICSM's scalability to larger projects.

1.1 The Challenge: Compressing Schedule by Adding Developers

From the beginning of software-time, people have wondered why it isn't possible to accelerate software projects by simply adding staff. This is sometimes known as the "why can't we make a baby with nine women in one month?" problem. The most famous treatise declaring this to be impossible is Fred Brooks' book in 1975, "The Mythical Man-Month", in which he declares that "adding more programmers to a late software project makes it later" and indeed this has proven largely true over the decades. However, since 1975 a number of strategies for accelerating systems and software development schedules have been found. A DoD Systems Engineering Research Center study[3] categorized these into product, process, project, and people strategies, which led to the development of a quantitative model for estimating the effects of the strategies on a project's schedule[4]. This study identified a number of good sources [5–13] for such strategies. One particular strategy for schedule acceleration with high payoff potential but relatively little application was to organize projects in ways that enabled developers to work in parallel, within a framework where the components were ensured to successfully interoperate. The following sections describe our approach to successfully execute this strategy on several relatively different applications.

1.2 Resilient Agile VS. Other Agile Methods

Resilient Agile has some commonalities and some differences with other agile approaches. Like most agile methods, RA "gets to code early" and uses feedback from executable software to drive requirements and design. Specifically, RA uses technical prototyping as a risk-mitigation strategy, for user interface refinement, to help "sanity-check" requirements for feasibility, and to evaluate different technical architectures and technologies. In our student projects, we have used technical prototyping to help develop background-capable, power-safe geofencing, to compare multiple voice-recognition toolkits, and to evaluate different toolkits for developing augmented reality applications.

Unlike many agile methods, RA does not support "design by refactoring", nor does it drive designs from unit tests. Instead, RA uses a minimalist UML-based design approach (Agile/ICONIX)[14] that starts out with a domain model to facilitate communication across the development team, and partitions the system along use case boundaries, which enables parallel development. Within each use case, RA follows a standard set of steps including a complete sunny-day/rainy-day description for all use cases (confronting rainy-day scenarios early adds resilience to software designs). Each use case is then "disambiguated" using a conceptual Model-View-Controller description, and then carefully designed. Typically, designs are shown on sequence diagrams; however, TDD may be used as an alternate detailed design process. Requirements are modeled and allocated to use cases, and traceability between requirements and design is verified during design reviews, further increasing resilience of the software.

Having a small, standardized set of design steps and artifacts facilitates communication among team members who are working in parallel and also makes developers more interchangeable. One of the characteristics of student projects that run across multiple semesters is nearly 100% staff turnover every 3 months since students typically don't take the same class in multiple semesters. This turnover rate has not caused a problem on our student projects.

Combining a UML-design approach with extensive prototyping allows RA to be both feedback-driven and plan-driven at the same time. Since both underplanning and overplanning are expensive, this blend of planning and feedback gets us near the cost minimum (Figure 1), and this is reflected in the actual effort expended on our student projects.

Having a UML model allows RA to leverage automation to further accelerate development. Domain models are made executable by code generation of database collections, CRUD functions and RESTful APIs. RA drives test plans from the design artifacts (Design Driven Testing)[15] and puts greater emphasis on acceptance testing as opposed to unit testing.

Resilient Agile does not specify a management process for delivering code but focuses strictly on the engineering design of the software. RA is flexible with respect to management process. It's possible to use RA with two-week Scrum sprints, but this process is not mandatory. If the sprint/backlog paradigm is desired, it's straightforward to populate the backlog with controllers from the MVC use case decompositions. On most of our student projects, we have worked with semester-long "sprints" but with developers working in parallel (one per use case) and each developer delivering either design artifacts or code on a weekly basis.

Finally, RA attempts to eliminate ceremony from the development process. Both agile and waterfall processes include significant amounts of ceremony (e.g. daily stand-up meetings) and ceremony costs time and money. For example, requiring all team members to participate in an hour-long standup meeting every day imposes a 12.5% tax (assuming an 8-hour workday) across the team.

However, working in parallel does require some form of organized communication between team members, as interfaces need to be negotiated. We have found that one 90-minute meeting per week (class time) is beneficial in this regard.

2 LEVERAGING PARALLEL DEVELOPMENT - THE HIGH-PAYOFF KEY TO SCHEDULE COMPRESSION

Our experiments at USC CSSE with Resilient Agile began with an effort that we initially called "Massively Parallel Use Case Modeling",

where we partitioned a Location Based Advertising system into a large number of use cases and assigned one use case to each of 47 students in Prof. Boehm's CS577 Software Engineering class. Two students from that class subsequently were admitted to the PhD program at USC and are co-authors of this paper. The lead author was the project owner, architect, instigator, troublemaker, "Keeper of The Holy Vision" (KOTHV), and also the author of Agile Development with ICONIX Process[14].

The project originated as two homework assignments to give students a hands-on learning exercise with UML and use case driven development, with the first homework assignment called "build the right system" and the second assignment called "build the system right". Homework 1 (Build The Right System) consisted of each student identifying requirements, storyboarding screens, writing use case narratives, and disambiguating their use cases via "robustness diagrams" (conceptual model-view-controller decomposition of a use case), while Homework 2 (Build The System Right) consisted of detailed design using class diagrams, sequence diagrams, database schemas, and occasionally state machines. While grading Homework 2, the chief troublemaker got the bright idea of offering an extra credit assignment for students to write prototype code for their use case, and 29 out of 47 students accepted the challenge.

Our original expectations for this exercise were that we would wind up with a fairly detailed UML model (which we did) and not much in the way of working code. The expectation of a decent UML model came from a couple of decades of ICONIX "JumpStart" training workshops where it is standard practice to work a real industry project with multiple lab teams where each team worked on a different subsystem of the project. In those classes we typically limit to 3 lab teams per instructor, so whether this could be stretched to 47 parallel threads of development was an unknown quantity.

The expectation of "not much in the way of working code" was based on the presumption that it was a complete impossibility to successfully integrate the work of 47 developers working in parallel. This expectation was exceeded for reasons we explain below, and we presented the results at the USC Annual Research Review under the title of "Massively Parallel Use Case Modeling". The results of CS577 were interesting enough that we decided to continue the project using Directed Research students as scenario developers, and work continued over a summer session, and the following fall and spring semesters, once again with students working in parallel. At the end of the exercise, we had evaluated multiple technical approaches and refined the prototype code into a fully functional system. We then continued the experiment with two other student projects and began gathering productivity data on the various projects. This experimentation and data gathering is presently continuing with additional student projects.

3 KEY ELEMENTS REQUIRED TO SUPPORT PARALLELISM

3.1 Just Enough Planning

Software processes can be rated on a formality scale ranging from "feedback driven" (less formal) to "plan-driven" (more formal). In Balancing Agility and Discipline[16] the case is made that some parts of the system may be feedback-driven (e.g., the user interface) and that some parts of the system may be plan-driven (e.g., the security kernel), but that for most complete systems the extremes on either side of this scale are expensive, and that there is a cost-minimum somewhere in the middle. In Agile Development with ICONIX Process[14], the case is made that "just enough planning" in the form of a minimalist, use case driven approach gets close to this cost minimum, and in fact, Resilient Agile has its roots in Agile/ICONIX. For CS 577 students we simplified the approach to Build the Right System (i.e. get the requirements right) and Build the System Right (i.e. careful design).

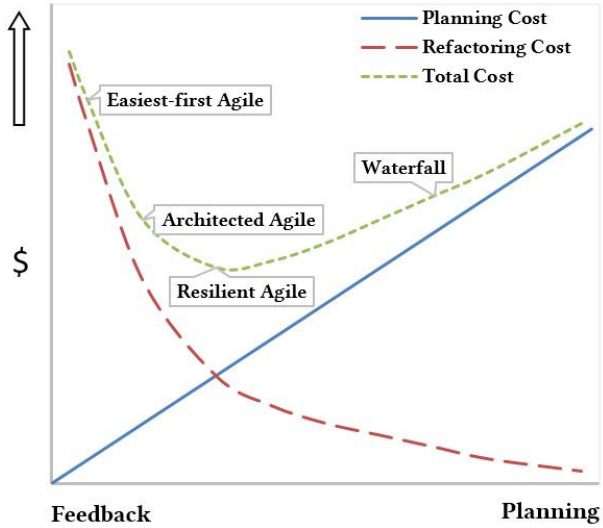

Figure 1: How much planning is "just enough"? We propose Build the Right System then Build the System Right.

3.2 Anticipation of Dependencies

When planning for parallel development, it's critically important to understand where dependencies between use cases exist, partition the project into appropriate subsystems, and ensure adequate collaboration and communication between developers whose work is interdependent. On the location based advertising (geofenced coupon delivery) project, we developed a cross-platform mobile app, a customer facing website, and a business-partner facing website which included coupon publishing and billing. A well-formed UML model with careful and continuous architectural review of independently developed designs was critical to a successful outcome.

3.3 Rapid and Flexible Database and API Development

One of the big reasons that we had low expectations for successful integration of independently developed student code was that it seemed to present an intractable database problem. With independent development, we had no right to expect that database schemas would be compatible. In this aspect we were aided by our choice of a NoSQL database rather than a relational database. NoSQL databases are very flexible by virtue of each record carrying its own JSON schema, and thus well-suited for "sandbox" development. Parallel development can then proceed in the sandbox, and when it reaches

maturity, the database schema can be unified and promoted out of the sandbox into a production instance of the database.

On the location based advertising project, the initial CS577 student team that developed the billing system had requested to work in PHP/MySQL because those students had more experience with that technology. Our success in other parts of the system using NoSQL was good enough that in the following semester we re-did this part of the system using Cassandra and JavaScript (JQuery Mobile) so that we had a unified code baseline. During this exercise we developed a design pattern where students could rapidly "clone" some sample database code that implemented the database "CRUD functions" (Create, Read, Update, Delete), and this proved successful enough that it became the seed for a student-developed database code generator that we describe below. We have piloted this database code generator on a subsequent RA project with excellent success.

3.4 Use of RESTful APIs (Microservice Architecture)

NoSQL databases did not exist when Brooks wrote Mythical Man Month. Neither did RESTful APIs and microservice architectures. These technologies are both helpful in integrating work performed by independent developers and development teams. In particular the ability to rapidly publish a RESTful API that makes it possible to create and read collections from a NoSQL database from both mobile apps and web apps allows for a great flexibility in development.

3.5 Prototyping Areas of Technical Risk

One of the luxuries of massively parallel development is that it's possible to have developers explore alternate technologies and mitigate risk early on, because you have lots of programmers available. This prototyping occurs in parallel, so you're not burning up months on the calendar to evaluate alternatives. On the location based advertising project we used this to our advantage by prototyping systems that used two different commercial geofencing products, in parallel with developing two homegrown geofencing plugins (one for Android and one for iOS). In the end our homegrown solution outperformed both commercial solutions. On a subsequent RA project (Bad Driver Reporting) multiple students prototyped voice activation frameworks until we found one that worked, and prototyped dual map/video display code for both iOS and Android. Meanwhile other parts of the project continued without waiting for these prototyping threads to complete.

3.6 Better Communication through Visual Models

One of the major factors behind Brooks' Law was that bringing new programmers onto a project placed an additional communication burden on the more experienced developers, so that schedule sometimes slipped because those who knew what was going on spent too much time training the newcomers.

Visual modeling was another technology that didn't exist in the days of OS 360. All of our student projects experienced nearly 100% personnel turnover every 3 months (occasionally we get lucky with a few returning students but for the most part it's a new class coming in every semester). We had great success handing the UML models from the previous semester to the previous students, who were then able to pick up the work without skipping a beat.

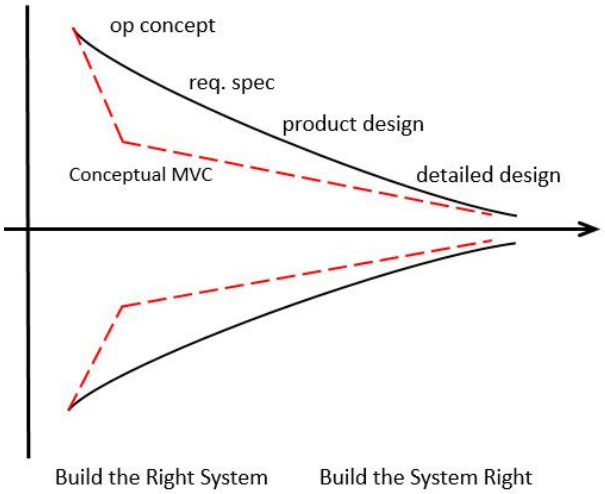

Figure 2: Pinching the cone of uncertainty at op concept reduces development time and cost.

Specific visual modeling techniques that are especially helpful include:

3.6.1 Domain Modeling. Starting the project with a domain model (dictated by the Keeper Of The Holy Vision: KOTHV) starts everybody with a common shared understanding of the problem domain and the objects in it.

3.6.2 Disambiguating use cases using storyboards and "conceptual MVC" models (robustness diagrams). These techniques allow the KOTHV to maintain a "mind-meld" with the developer and know with confidence that what is being built will match the Holy Vision. One of the most useful benefits of this is avoidance of scope-creep. Developers have a tendency to go off on their own and write code for which there are no requirements, but reviewing the conceptual designs for each use case stops this process in its tracks.

In the Incremental Commitment Spiral Model, Boehm introduces a concept called "the cone of uncertainty". The robustness / MVC / disambiguation technique in particular, combined with the other aspects of "Building the Right System", changes the shape of this cone of uncertainty for a use case. MVC decomposition of a use case reduces the uncertainty for that use case by forcing an explicit breakdown into screens, data collections, and high level functions, and refining the use case narratives to be more explicit and less ambiguous.

Applying this technique across all use cases of a system (in parallel) pinches the neck of the cone of uncertainty at the "op concept" stage of development, providing a more stable behavior specification on which to implement the various use cases, which can also proceed in parallel. In contrast, many agile processes disambiguate use cases by producing working code, accompanied by a suite of unit tests (a labor intensive process when compared to a quick MVC

decomposition). When projects are under planned, this can often be expensive as refactoring costs will be high.

3.7 Continuous Testing in Parallel with Development

When you have a large pool of developers available it's not too difficult to task some of them with starting to test prototype code as it gets developed, thus providing continuous feedback to developers. This is particularly useful when prototyping technologies such as background-friendly geofencing, which, for a mobile coupon delivery system, have stringent performance requirements and also physical requirements such as power consumption (achieving high-accuracy with geofencing requires use of power-hungry GPS circuitry). Resilient Agile incorporates techniques described in Design Driven Testing[15], which drives test planning from a variety of model artifacts such as use cases. Having the system engineers (KOTHV), developers, and testers work concurrently rather than sequentially is another example of project speedup via parallel performance.

4 DATABASE CODE GENERATION - MOVING RAPIDLY FROM DOMAIN MODEL TO EXECUTABLE PROTOTYPE

4.1 Executable Domain Models

Domain modeling has long been one of the keys to success when following a use case driven process. Writing the use cases in a common vocabulary defined by the domain model is essential to smooth collaboration when multiple developers are implementing use cases in parallel. This has been true for many years and is independent of any particular database technology because it relates to "Building the Right System" (i.e. getting the behavior requirements straight). Executable domain models in the form of NoSQL collections and REST APIs that implement database access functions offer even greater advantages when it comes to developers working in parallel sandboxes and later integrating their work into a cohesive whole.

The first advantage is that if you can think of something that belongs in the problem domain you can immediately make it live in the database. In essence, as a developer you can simply start calling methods to create and manipulate conceptual entities. A second advantage is that when multiple use cases share the same domain object (for example a BadDriverReport object), it's not necessary to predefine all of the attributes for that object. This is helpful because it's usually the case that different use cases will need different attributes. Each developer can start building immediately with the attributes they need, and worry about integration later. All that's required is that the correct name of the domain object is used by all developers who need that object.

In Resilient Agile, the conceptual domain model is processed as input for generating the database backend and RESTful API. This happens very early in the project, enabling developers to work in parallel with a live database immediately. The general process consists of two steps:

4.1.1 Extraction. A domain model contains all the entities involved in the proposed system. It also shows the attributes and behaviors for each entity. The code generation process will first extract this information from the domain model by analyzing its profile.

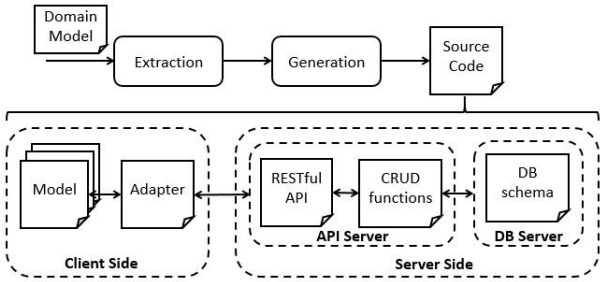

Figure 3: Domain Models become executable early in the project via code generation.

4.1.2 Generation. The code generator then processes and organizes the information. On the client side, it generates corresponding models and wraps basic functions in an adapter for utilizing. On the server side, it provides DB schema and basic CRUD functions for data queries, and basic RESTful APIs for resource accessing.

4.2 Instant Microservice Architecture

Microservice architecture has become increasingly popular in today's software development. Services can be accessed via URL using RESTful APIs. Using microservice architecture, conceptual MVC decompositions of use cases can be quickly realized.

Our goal is to accelerate project progress by rapid integration and release of what has been designed. At the very beginning, a draft design of the domain model can be directly translated into source code. The source code provides the whole backend, which covers both server side and client side, for the developers to access. At any point in time, when the domain model evolves, the entire backend can be regenerated immediately. In this way, database code generation enables true evolutionary development (i.e. it's true to the goals of agility, whether or not we use Scrum and TDD).

4.3 Databases Targeted

It's clear that some projects will find a NoSQL model most effective, but others will find a SQL model to be a better fit to their usage patterns. Some might need both for different parts of their business. Moreover, there are many flavors of database whatever model is chosen. For the above reasons, in order to be most effective across a wide range of projects, the code generation should be re-targetable towards a variety of databases. The decision varies from project to project.

To help make the best choice, several criteria should be taken into consideration:

4.3.1 Scalability. If the system tends to contain big data, or to be massive with a large number of users, the NoSQL model should be considered.

4.3.2 Flexibility. If the data schema change frequently in order to better fit the requirements, or tends to be complex, the NoSQL model should be first considered.

4.3.3 Functionality. If the database is expected to serve complex queries, the SQL model should be chosen.

4.3.4 Relation Complexity. If there are complex relationships existing between different sets of data, SQL model's join capability would be the better choice.

When attempting schedule compression via massively parallel development, flexibility is key. The data schema changes frequently because individual developers are working in "sandbox mode" and may require different attributes on the database collections. Of course, attributes must be merged when moving out of the sandbox into a production system. Because flexibility is so important, current research has focused on NoSQL databases, with all scripts and generated code written in JavaScript. Specifically the initial implementation of the code generator generates code compatible with Mongo DB and Node JS (i.e. MEAN stack). Future versions of the code generator will target additional databases.

4.4 Adapting to Changing Requirements via Database and API Regeneration

Responding to rapidly changing requirements is one of the key reasons that organizations choose to adopt agile processes. In a more formal agile process we first refactor the design based on new requirements, then implementation follows. Large amounts of time may be spent refactoring. In less formal agile processes, code is modified without documentation, which makes the code mismatch the design. The mismatch is untraceable and will lead to a series of other problems.

With RA we can instantly regenerate the backend (both database and REST API) from the domain model, giving us a way to incrementally and quickly prototype the design. Design changes to accommodate shifting requirements are reflected in the design. In this way, executable domain models enable evolutionary development within the context of a model-driven project.

5 SUMMARY OF RESILIENT AGILE STUDENT PROJECTS TO DATE

Our experimentation with RA at the USC Center for Software Engineering started with a Location Based Advertising (LBA) project. This project consisted of an initial team of 47 students in a Masters level Software Engineering class as two homework assignments and was continued over 3 semesters using different teams of Directed Research (DR) students. DR students are given a time budget of 5 hours per week per unit, and most students take only 1 unit per semester. Approximately 75 students overall worked on the LBA project, with a 90% staff turnover every 3 months. During this time we did extensive prototyping of different geofencing solutions, and introduced a test-team operating concurrently with the developers to evaluate the performance of the various prototypes and conduct in-the-field acceptance testing.

Our next attempt was with a photo-sharing app called PicShare, where we originally wanted to implement the same project with two independent teams, with one using Architected Agile and one using RA. The productivity comparison effort was hampered by the RA team initially getting some incorrect guidance on how to do MVC decomposition which required extensive re-work by the 10 student RA team. However, once the team got straightened out they produced excellent results.

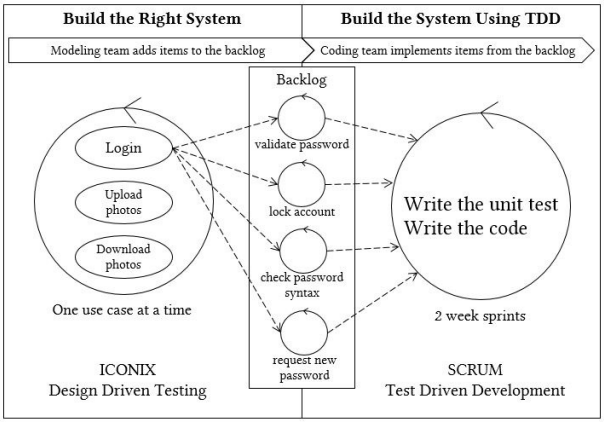

Figure 4: Resilient Agile is compatible with a Scrum/TDD implementation but does not mandate one.

The third experiment was with a Crowdsourced Bad Driver Reporting System (BDR) which involved 15 students working in parallel for approximately 12 weeks at 5 hours per week per student. This was the first project where we deployed the database code generator. The BDR team developed a proof-of-concept system consisting of voice-activated "dashboard camera" mobile apps for iOS (Swift) and Android (Java), and a web app implemented in Angular JS for filing, reviewing and querying video-centric bad driver reports. This project is about to be continued by a new student team who will move it from proof-of-concept into a more robust system.

A new augmented-reality game project will also be attempted in the upcoming semester.

6 RESILIENT AGILE AND SCRUM/TDD

RA is a flexible process in that development can proceed using the now ubiquitous Scrum-sprint-backlog paradigm or each developer can design and implement the use case they have specified. The authors are fully aware that many people will not regard any process as "agile" unless it has Scrum and TDD in it.

With the Scrum-sprint approach, modeling is focused on requirements and conceptual design, then controllers from the MVC decompositions are added to the backlog, and TDD can be used as an alternative design approach to UML sequence and class diagrams. Once the use cases have been elaborated through conceptual design which includes both sunny-day and rainy-day behavior descriptions, the detailed design process can be left to the discretion of the Scrum teams with little risk of scope creep. Our work on student projects to date has not used the Scrum/TDD development strategy, as we have focused on compressing schedules using parallelism. This remains an area for future exploration and data collection.

Table 1: Effort estimation by Early Use Case Point (EUCP)

Project	UEUCW[1]	UAW[2]	TCF[3]	ECF[4]	EUCP[5]	Estimate (Man-hour)	Actual (Man-hour)
BDR	125	14	1.14	1.21	191.74	1593.58	1392
PicShare	200	8	1.06	1.03	227.09	1793.31	2016
LBA	370	12	1.12	1.32	564.75	3701.09	3680

Table 2: Effort estimation by Extended Use Case Point (EXUCP)

Project	UEXUCW[6]	UAW	TCF	ECF	EXUCP[7]	Estimate (Man-hour)	Actual (Man-hour)
BDR	62	14	1.14	1.21	104.83	1523.78	1392
PicShare	132	8	1.06	1.03	152.85	1858.96	2016
LBA	270	12	1.12	1.32	416.91	3702.10	3680

7 ESTIMATION OF RESILIENT AGILE

To apply RA in a corporate rather than a student environment, it will be important to avoid understaffing or overstaffing the number of developers needed. This made it important to come up with an effort estimation model that could be used for corporate RA projects, while being calibrated to the actual effort on the student projects. An initial cost model for such corporate RA projects has been developed and calibrated, with reasonable accuracy. It is actually a series of three models, based on the information that will be available at the three decision points: Early Use Cases, Extended Use Cases, and Function Point Sizing. Tables 1, 2 and 3 provide a summary of the inputs, estimates, and actuals for the 3 projects. The inputs include not just sizing parameters, but also first-order technical complexity factors and environmental complexity factors, roughly comparable to the function point General System Characteristics. Future extensions will provide more detailed rating scales for these, and comparisons with estimates based on other models[17–25].

7.1 Early Use Case Points (EUCP) enable Predictive Complexity Estimation

Because RA projects follow a well-defined sequence of modeling steps before coding begins, and because these models are captured electronically, we can mine the models using scripts for information that is useful in predicting size and cost drivers before coding. Ideally we would like to be able to get rough estimates during the "build the right system" (requirements exploration) phase, and then refine our estimates during the "build the system right" (design) phase. Because RA is use case driven at its core, we have started from previous work on Use Case Points and extended the concept to include a first estimate deriving Early Use Case Points (EUCP) from cyclomatic complexity of use case paths, and then a more detailed estimate (driven from MVC decompositions) based on Transactional Complexity within the use cases. Finally during detailed design we can derive more accurate estimation by counting model elements such as the number of messages on a sequence diagram.

Early Use Case Points (EUCP) can be derived by analyzing the number of sunny-day and rainy-day paths through a use case (typically by elaborating the use case on an activity diagram) and computing cyclomatic complexity of the activity diagram. Since system requirements are very fluid at this early stage of the process it's important to be able to generate these first-order estimates quickly.

[1]UEUCW: Unadjusted Early Use Case Weight
[2]UAW: Unadjusted Actor Weight
[3]TCF: Technical Complexity Factor
[4]ECF: Environmental Complexity Factor
[5]EUCP: Early Use Case Point

Fortunately a substantial amount of automation exists for generating activity diagrams from use case descriptions in the Enterprise Architect CASE tool, using automatic activity diagram generation from "structured scenarios". This approach is detailed in Design Driven Testing[16]. The estimation process follows the steps below:

- Convert Use Case narratives to structured scenarios.
- Convert structured scenarios to activity diagrams.
- Count the cyclomatic complexity of each activity diagram.
- Based on the cyclomatic complexity of each activity diagram, count the Unadjusted Early Use Case Weight (UEUCW) for each identified use case.

7.2 Extended Use Case Points (EXUCP) measure Transactional Complexity within Use Cases

In RA, use cases are decomposed as conceptual-MVC (robustness) diagrams which break each use case into Model (entity), View (boundary), and Controller (control) elements. Controllers on the robustness diagram represent transactions within a use case. This process can be automated using scripts on the UML models.

We can compute transaction weight using the following steps:

- Identify the transactions by counting controllers on robustness diagrams.
- Count the number of UI elements (boundary elements) involved in each identified transaction.
- Count the number of domain objects (entity objects) involved in each identified transaction.
- Evaluate the complexity level for each transaction.

Using these methods we can automatically generate a use case complexity estimate. Note that this method is predicated on users producing well-formed robustness diagrams. This presented a practical difficulty on our second RA project (PicShare) when the team received some incorrect mentoring guidance instructing them to show system behavior on associations rather than using controllers. In general the conceptual MVC technique for elaborating use cases is not as well understood as techniques such as activity and sequence diagrams. An additional cost driver to account for the use of the automatic code generator use on the BDR project reduced its effective size, and explained how the 15 BDR developers could use less effort than the 10 PicShare developers.

[6]UEXUCW: Unadjusted Extended Use Case Weight
[7]EXUCP: Extended Use Case Point

Table 3: Effort estimation by Application Function Point (AFP)

Project	AFP[8]	VAF[9]	AFPC[10]	Estimate(Man-hour)	Actual(Man-hour)
BDR	403	1.16	467.48	1251.15	1392
PicShare	714	1.14	813.96	2273.26	2016
LBA	1054	1.19	1254.26	3572.15	3680

7.3 Application Function Point (AFP) Estimation

Sequence and class diagrams are available by elaborating the behavioral and structural aspects of the system in UML. With information available at this level of detail, we can count actual Function Points for each use case, following the method suggested in the study of measuring Function Points by UML Design Specification[26]. This provides a means to achieve the goal of fast measurement of function points.

7.4 Cost and Schedule Estimation Summary

The overall effort numbers for Location Based Advertising (3680 hours) add up to a little under 2 man-years, distributed across approximately 75 students over several semesters, to build a fully-functional system. It was this result that sparked our interest and has led to further study of how to leverage parallelism in development to reduce cost and compress schedules.

One of the main drivers behind Resilient Agile is the compression of schedule and reduction of overall cost of system development. Thus it is important to obtain unbiased and objective cost and schedule data on RA projects. This work is in a very early stage as we have only been able to study a small number of RA projects to date, and because the first RA project (Location Based Advertising) was mostly complete before we started the estimation work.

Since we are introducing a database and REST API code generator as part of the RA process, it's important to quantify the effect of "executable domain models" on development. As seen in Table 1, a cost driver for automatic code generation was added to reduce the effective size of BDR, as is often done to account for other effort reduction strategies such as code reuse.

Finally the RA effort estimation model provides an opportunity to develop predictive corporate RA estimates, and to use these estimates to help guide project management in allocating development resources for development.

8 SCALABILITY AND COMPATIBILITY WITH THE INCREMENTAL COMMITMENT SPIRAL MODEL (ICSM)

The Resilient Agile (RA) process is consistent with the four principles of the incremental Commitment Spiral Model (ICSM):

- Stakeholder value-based guidance.
- Incremental commitment and accountability.
- Concurrent multi-discipline engineering.
- Evidence and risk-based decisions.

Stakeholder value-based guidance is built up via continuing involvement of stakeholders in iterating and discussing storyboards, prototypes, analyses, constraints, and concepts of operation. *Incremental commitment and accountability* is built up via periodic stakeholder reviews of the system's increasingly complete definition, prototypes, future plans, and resource requirements. *Concurrent multi-discipline engineering* is built up via participation of stakeholder experts, and often of experts from second-order or third-order stakeholders. *Evidence and risk-based decisions* are made as stakeholders see that shortfalls in evidence are probabilities of stakeholder loss, which when multiplied by size of loss becomes Risk Exposure. Sometimes stakeholders will accept that the higher risk is worth the benefits of early system fielding, and decide to go forward, but in other cases will decide that obtaining further evidence of system feasibility is preferable.

Such risks are encountered in Pure Agile, in which developers choose easiest-first initial capabilities, This often achieves early user satisfaction, but often different users and developers make incompatible usage and implementation choices, leading to extensive refactoring and delays. The Architected Agile[1]approach addresses this problem, but at the expense of a longer startup.

RA can be considered as a new common case of the ICSM table of common process cases in Chapter 11 of the ICSM book[2], p.202. The RA common case is similar to the Architected Agile common case, with an additional criterion of a High-Very High premium on Rapid Fielding and Modification, and wider ranges on the criteria for Size/Complexity (Low to Very High) and Criticality (Low to Very High), giving RA the advantages of rapid development and high assurance.

The keys to RA superiority are its ability to enable many developers to work in parallel, and the three-team approach to have a continuing system engineering team handling the change traffic and emerging risks and stabilizing the developers, and the continuing independent verification and validation (V&V) team quickly finding problems for quick resolution and reducing technical debt, as shown in Figure 5 from[2], p. 245.

For two out of the three small and very quick and well-V&Ved RA projects summarized above, there was a single "keeper of the holy vision" performing the continuing systems engineering function, creating the architecture supporting multiple parallel developers, and working out stakeholder-coordinated feature content. There were relatively small but well-qualified V&V teams performing continuous testing.

To scale up the RA approach to large projects, alternative approaches for system engineering, development, and V&V can be applied. An excellent example is the million-line, 3-increment TRW Command Center Processing and Display System-Replacement (CCPDS-R) project summarized in[2], pp. 101-103, and described in detail in[27], pp. 299-362. Instead of a single system engineer, there was a team averaging around 10 system engineers working out an architecture enabling many programmers to work in parallel, not only developing Ada package specifications for the various module, but also having them consistency-checked by the Ada compiler.

[8]AFP: Application Function Point
[9]VAF: Value Adjustment Factor - VAF is the summarized value for the 14 GSCs by IFPUG counting standard
[10]AFPC: Adjusted Function Point Count

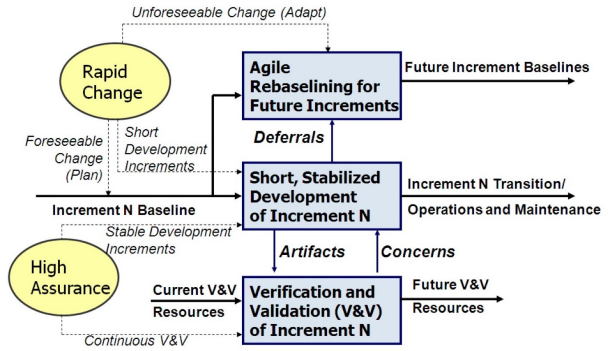

Figure 5: RA is compatible with ICSM's Evolutionary-Concurrent Rapid Change Handling and High Assurance approach.

This ensured that a development team averaging around 75 programmers could work in parallel with minimal coordination delays. The V&V team averaging around 10 V&Vers were aided by having developed an executing architectural skeleton, enabling developers to replace dummy modules satisfying the package specifications by programmed modules for continuous testing. These aids to rapid and high-assurance development were therefore an early example of the RA approach scaling up to a very large, reliability-critical project following the ICSM principles and practices.

In principle, the RA approach is scalable to perform continuous delivery[28], using techniques such as Lean Development[29], Kanban[6], Scaled Agile[30], and DevOps[31]. However, these approaches often have degrees of freedom that cause violations of the critical success factors (CSFs) employed in the CCPDS-R project. These factors were identified by Siegel[32], who analyzed the failure sources of previous large TRW projects, extended the set of CSFs, and successfully led the delivery of several even-larger TRW projects. The main CSF is to have an architecture that specifies every independently-schedulable software entity within the system, and determines how they will successfully interact. If developers have too much freedom and add incompatible independently-schedulable entities, the results have been serious performance and integrity problems.

9 CONCLUSIONS

We have described an ongoing series of experiments in achieving schedule compression by exploiting parallelism in development. The process we have followed (Resilient Agile) includes extensive prototyping to mitigate technical risk, a minimalist UML-based process (Agile/ICONIX), and a code generator that enables "executable domain models" to be rapidly realized with a NoSQL database (Mongo DB) and a REST API (Node JS) that supports common database functions.

The experiments have been conducted with more than 100 Masters Computer Science students over a 3 year period on 3 projects to-date, and are still ongoing. The class projects have mostly been "directed research" students with a time budget of around 5 hours per week for a 3 month semester, and team sizes ranging from 10 to 47 students working in parallel. We have tried to do "just enough planning" to ensure that work developed independently by large numbers of students working in parallel integrates successfully, and in fact, integration has not proven to be a problem on any of the projects attempted.

Empirical results from the projects attempted (e.g. less than 2 man-years total effort for 75 students to build a complete Location-Based Advertising system) have been encouraging and detailed effort tracking and productivity measurements are being deployed on new projects in an attempt to quantify the empirical results more precisely. More data is being gathered to facilitate accurate calibration of cost models.

More generally, the USC and CCPDS-R projects identify some generally underutilized key enablers for speeding up software deliveries by enabling project participants to operate in parallel rather than sequentially. These include:

- Well-defined interface definitions between system components (REST APIs for the USC projects; Ada package specs for CCPDS-R);
- Having systems engineers, developers, and testers operating concurrently rather than sequentially;
- Just Enough Planning (balancing the costs and risks of doing too little vs. too much);
- Pinching the "cone of uncertainty" during the early op concept phase, using a mix of prototyping to evaluate technical alternatives and MVC decompositions to disambiguate behavior specs;
- Leveraging code generation of databases and APIs to rapidly respond to changing requirements;
- Organizing deliveries to be incremental or continuous;
- Getting project personnel and their other stakeholders to agree to the approach and to keep each other informed about impending changes;
- Providing rewards to participants for continuous performance and providing useful improvement suggestions.

REFERENCES

[1] Barry Boehm, Jo A. Lane, Supannika Koolmanojwong, and Richard Turner. 10.4.1 architected agile solutions for software-reliant systems. *INCOSE International Symposium*, 20(1):1289–1302, July 2010.
[2] Barry W. Boehm, Jo A. Lane, Supannika Koolmanojwong, and Richard Turner. *The incremental commitment spiral model: principles and practices for successful systems and software.* Addison-Wesley, Upper Saddle River, NJ, 2014.
[3] Debra F Lepore and John Colombi et al. "expedited systems engineering for rapid capability and urgent needs", final report. *SERC TR-034*, December 2012. http://sercuarc.org.
[4] Dan Ingold, Barry Boehm, and Supannika Koolmanojwong. A model for estimating agile project process and schedule acceleration. In *Proceedings, ICSSP 2013*, pages 29–35, New York, NY, USA, 2013. ACM.
[5] J.S. Ford, R.M. Colburn, and Y.A. Morris. *Principles of Rapid Acquisition and Systems Engineering.* Air Force Institute of Technology, 2012.
[6] D.J. Anderson. *Kanban: Successful Evolutionary Change for Your Technology Business.* Blue Hole Press, 2010.
[7] L.J. Arthur. *Rapid Evolutionary Development: Requirements, Prototyping & Software Creation.* Wiley Series in Software Engineering Practice. John Wiley & Sons, 1992.
[8] J.A. Highsmith. *Adaptive Software Development: A Collaborative Approach to Managing Complex Systems.* Dorset House Pub., 2000.
[9] S. McConnell. *Rapid Development.* Developer Best Practices. Pearson Education, 1996.
[10] E. Murman, T. Allen, K. Bozdogan, J. Cutcher-Gershenfeld, H. McManus, D. Nightingale, E. Rebentisch, T. Shields, F. Stahl, M. Walton, et al. *Lean Enterprise*

Value: Insights from MIT's Lean Aerospace Initiative. Palgrave Macmillan UK, 2016.

[11] J.P. Womack and D.T. Jones. *Lean Thinking: Banish Waste And Create Wealth In Your Corporation.* Simon & Schuster UK, 2013.

[12] K. Beck and C. Andres. *Extreme Programming Explained: Embrace Change.* The XP series. Addison-Wesley, 2005.

[13] M.A. Cusumano and R.W. Selby. *Microsoft Secrets: How the World's Most Powerful Software Company Creates Technology, Shapes Markets, and Manages People.* A Touchstone book. Free Press, 1998.

[14] D. Rosenberg, M. Collins-Cope, and M. Stephens. *Agile Development with ICONIX Process: People, Process, and Pragmatism.* ITPro collection. Apress, 2006.

[15] M. Stephens and D. Rosenberg. *Design Driven Testing: Test Smarter, Not Harder.* Books for professionals by professionals. Apress, 2011.

[16] B.W. Boehm and R. Turner. *Balancing Agility and Discipline: A Guide for the Perplexed.* Addison-Wesley, 2004.

[17] Barry W. Boehm. *Software cost estimation with Cocomo II.* Prentice Hall, Upper Saddle River, NJ, 2000.

[18] Barry W. Boehm. *Software engineering economics.* Prentice-Hall, Englewood Cliffs, N.J, 1981.

[19] S. Chulani, B. Boehm, and B. Steece. Bayesian analysis of empirical software engineering cost models. *IEEE Transactions on Software Engineering*, 25(4):573–583, 1999.

[20] Fan Wang, Xiaohu Yang, Xiaochun Zhu, and Lu Chen. Extended use case points method for software cost estimation. In *International Conference on Computational Intelligence and Software Engineering*, pages 1–5. IEEE, 2009.

[21] Roy K Clemmons. *Project Estimation with Use Case Points.* CrossTalk, February 2006.

[22] FJ F. Heemstra and RJ R. Kusters. Function point analysis : evaluation of a software cost estimation model. *European Journal of Information Systems*, 1(4):229–237, 1991.

[23] J. E. Matson, B. E. Barrett, and J. M. Mellichamp. Software development cost estimation using function points. *IEEE Transactions on Software Engineering*, 20(4):275–287, 1994.

[24] Mohammad Azzeh. Software cost estimation based on use case points for global software development. In *5th International Conference on Computer Science and Information Technology (CSIT)*, pages 214–218. IEEE, 2013.

[25] Kasi Periyasamy and Aditi Ghode. Cost estimation using extended use case point (e-ucp) model. In *International Conference on Computational Intelligence and Software Engineering*, pages 1–5. IEEE, 2009.

[26] T. Uemura, S. Kusumoto, and K. Inoue. Function point measurement tool for uml design specification. In *Proceedings, Sixth International Software Metrics Symposium*, pages 62–69, 1999.

[27] W. Royce. *Software Project Management: A Unified Framework.* Object Technology Series. Addison-Wesley, 1998.

[28] Jan Bosch. *Continuous software engineering.* Springer, 2014.

[29] C. Larman and B. Vodde. *Scaling Lean & Agile Development.* Addison-Wesley, 2009.

[30] D. Leffingwell. *Scaling Software Agility-Best Practices for Large Enterprises.* Addison-Wesley, 2007.

[31] J. Davis and K. Daniels. *Effective DevOps.* O'Reilly Media, Inc, 2016.

[32] Neil Gilbert Siegel. *Organizing Complex Projects Around Critical Skills, and the Mitigation of Risks Arising from System Dynamic Behavior.* PhD thesis, University of Southern California, Los Angeles, CA, USA, 2011.

[33] P.J. Sadalage and M. Fowler. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence.* Addison-Wesley Professional, 1st edition, 2012.

[34] Harshal D. Gurad and V. S. Mahalle. An approach to code generation from uml diagrams. *International Journal of Engineering Sciences & Research Technology*, 3(1), 2014.

[35] Anshul, Sompal, and Vikas Sheoran. Automatic code generation using uml to xml schema transformation, international journal of advancement in engineering. *Technology, Management & Applied Science*, 1(2), 2014.

[36] Wafa Chama, Raida Elmansouri, and Allaoua Chaoui. Model checking and code generation for uml diagrams using graph transformation. *International Journal of Software Engineering & Applications*, 3(6):39–55, 2012.